EEG Protocol Notes

InEar Teensy protocols

Optimized protocol saves ∼49% bandwidth by sending aux data only when it changes.

---

1 Quick Overview

Three EEG streaming protocols:

Table 1: Protocol overview comparison

	OpenBCI Cyton	**InEar Original**	**InEar Optimized**
Purpose	General-purpose 8ch EEG	In-ear 5ch EEG + biometrics	Same hardware, smarter protocol
Packet Size	33 B (fixed)	56 B (fixed)	26–55 B (variable)
Sample Rate	250 Hz	1000 Hz	1000 Hz
Baud Rate	115,200	2,000,000	2,000,000
Data Rate	8,250 B/s	56,000 B/s	∼28,500 B/s
Bandwidth Used	71.6%	28.0%	14.3%
Error Detection	None (relies on USB)	XOR checksum	XOR checksum
Sensors	8 EEG + accel	5 EEG + accel + PPG + temp + batt + sync	Same sensors, sent at native rates

2 The Three Protocols

2.1 OpenBCI Cyton

Wireless EEG. Cyton board → USB dongle over 2.4 GHz radio. Shows up as virtual serial port.

- **8 EEG channels** at 24-bit resolution (ADS1299)
- **3-axis accelerometer** (LIS3DH)
- **250 Hz** sample rate (limited by radio bandwidth)
- **Wireless** link (2.4 GHz Nordic Gazelle protocol)
- **No checksum** — relies on USB's built-in error detection
- Max radio payload is 31 bytes; dongle adds 2 framing bytes → 33 bytes

2.2 InEar Teensy Original

Wired USB. Teensy 4.1 + ADS1299 + biometric sensors. In-ear EEG device.

- **5 EEG channels** at 24-bit resolution
- **9 auxiliary channels**: accelerometer (3), PPG heart sensor (3), temperature, battery, sync
- **1000 Hz** sample rate
- **Fixed 56-byte packets** — every packet carries ALL sensor data
- XOR checksum for error detection

2.3 InEar Teensy Optimized

Same hardware, different protocol. Sends each sensor at its own rate, not everything at 1 kHz:

- **Variable-length packets** (26–55 bytes) based on content
- **Multi-rate sampling**: EEG at 1000 Hz, accelerometer at 250 Hz, PPG at 100 Hz, temperature/battery at 10 Hz
- **Enumerated packet types** tell the receiver exactly what data is included

- **Sample-and-hold**: receiver keeps the last value for sensors not in the current packet
- Same error detection (XOR checksum)

3 What The Packets Look Like

3.1 OpenBCI Cyton — 33 Bytes

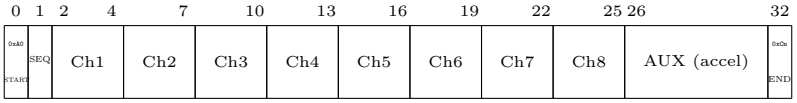

Figure 1: OpenBCI Cyton packet structure (33 bytes)

- **START** (1 B): Sync marker, always `0xA0`
- **SEQ** (1 B): Packet counter (0–255), detects lost packets
- **EEG** (24 B): 8 brain signal readings, 24-bit signed big-endian each
- **AUX** (6 B): Accelerometer X, Y, Z (or timestamps)
- **END** (1 B): Stop byte (`0xC0`–`0xC6`), also encodes packet type

Note to self: No checksum. Relies on USB error checking. Wireless link = slower baud, fewer channels.

3.2 InEar Teensy Original — 56 Bytes

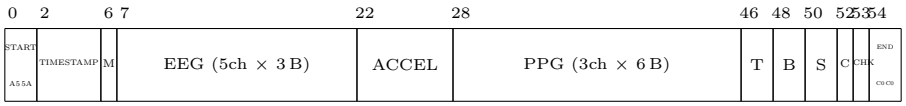

Figure 2: InEar Original packet structure (56 bytes). M=Marker, T=Temp, B=Battery, S=Sync, C=Counter.

Field	Size	Description
START	2 B	Sync marker: `0xA5` `0x5A`
TIMESTAMP	4 B	Sample time in microseconds
MARKER	1 B	Experiment event tag
EEG	15 B	5 channels, 24-bit signed big-endian
ACCEL	6 B	3-axis accelerometer (X, Y, Z)
PPG	18 B	3 channels × 48-bit (Red, IR, Green)
TEMP	2 B	Body temperature
BATTERY	2 B	Battery voltage
SYNC	2 B	External trigger signal
COUNTER	1 B	Packet counter (0–255)
CHECKSUM	1 B	XOR of all preceding bytes
END	2 B	Footer: `0xC0` `0xC0`

Table 2: InEar Original field descriptions

Note to self: Every packet has all sensors, even slow ones like temp/battery.

3.3 InEar Teensy Optimized — 26 to 55 Bytes

Key idea: skip data that hasn't changed.

Signal	Rate	Send interval	Justification
Brain (EEG)	1000 Hz	Every packet	Fast-changing neural signals
Motion (Accel)	250 Hz	Every 4th packet	Medium-speed head movement
Heart pulse (PPG)	100 Hz	Every 10th packet	Slow cardiac rhythm
Temperature	10 Hz	Every 100th packet	Near-DC signal

Table 3: Multi-rate sensor scheduling in the optimized protocol

Most Common Packet — EEG Only (26 bytes)

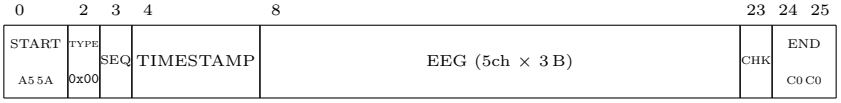

Figure 3: EEG-only packet (Type `0x00`) — 749 per second

Every 4th Packet — EEG + Accel (32 bytes)

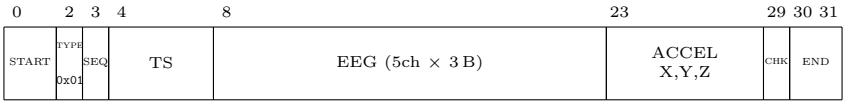

Figure 4: EEG + Accelerometer packet (Type `0x01`) — 200 per second

Full Sync Packet — Everything (54 bytes)

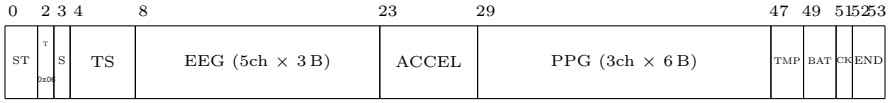

Figure 5: Full sync packet (Type `0x06`) — 1 per second

Type	Contents	Size	Per second
`0x00`	EEG only	26 B	749
`0x01`	EEG + Accel	32 B	200
`0x02`	EEG + PPG	44 B	40
`0x03`	EEG + Accel + PPG	50 B	9
`0x04`	EEG + Health	30 B	1
`0x05`	EEG + Accel + Health	36 B	∼0
`0x06`	Everything	54 B	1
Add `0x10` to any type → also includes experiment marker			

Table 4: Optimized protocol packet type reference

4 What's Actually In Each Packet

The three protocols carry different data. Bandwidth comparison alone is misleading.

4.1 Side-by-Side Sensor Comparison

Data Type	OpenBCI Cyton	InEar Original	InEar Optimized
EEG Channels	8	5	5
EEG Resolution	24-bit	24-bit	24-bit
EEG Bytes/Sample	24	15	15
Accelerometer	3-axis, 16-bit	3-axis, 16-bit	3-axis, 16-bit
PPG (Heart)	✗ None	3ch × 48-bit (18 B!)	3ch × 48-bit
Temperature	✗ None	16-bit	16-bit
Battery	✗ None	16-bit	16-bit
Sync/Trigger	✗ None	16-bit	✗ Removed
Marker	Via footer byte	1 byte	1 byte (optional)
Timestamp	Via aux (optional)	4 bytes	4 bytes

Table 5: Sensor data carried by each protocol

4.2 Useful Payload Per Sample

Protocol	EEG	Aux	Payload	Overhead	Ovh. %
OpenBCI Cyton	24 B	6 B	**30 B**	3 B	9%
InEar Original	15 B	30 B	**45 B**	11 B	20%
InEar Opt. (EEG-only)	15 B	0 B	**15 B**	11 B	42%
InEar Opt. (avg)	15 B	∼13.5 B	**∼28.5 B**	∼11 B	∼28%

Table 6: Payload vs. overhead per sample

4.3 The PPG Problem

PPG (heart pulse) uses the most bandwidth:

$$\text{PPG} = 3 \text{ channels} \times 6 \text{ bytes} = 18 \text{ bytes per sample}$$

More than all 5 EEG channels combined (15 bytes).

In the original protocol: 18 bytes × 1000/sec = 18,000 B/s for PPG alone.

PPG content goes up to ∼25 Hz. Nyquist minimum is ≥50 Hz. Optimized protocol sends at 100 Hz (2× margin):

$$\begin{aligned} \text{Original PPG:} &\quad 18\,\text{B} \times 1000\,\text{Hz} = 18{,}000\,\text{B/s} \\ \text{Optimized PPG:} &\quad 18\,\text{B} \times 100\,\text{Hz} = 1{,}800\,\text{B/s} \\ \text{Savings:} &\quad 16{,}200\,\text{B/s (90\% reduction for PPG alone!)} \end{aligned}$$

4.4 If OpenBCI Had The Same Sensors

If OpenBCI Cyton had to send the same data as InEar at 1000 Hz:

$$\begin{aligned} \text{Needed per sample:} &\quad 15 + 6 + 18 + 2 + 2 + 5 = 48\,\text{bytes} \\ \text{Needed per second:} &\quad 48 \times 1000 = 48{,}000\,\text{B/s} \\ \text{Available (8-N-1):} &\quad 115{,}200 \div 10 = 11{,}520\,\text{B/s max} \end{aligned}$$

Note to self: Would need 4.2× available bandwidth. Not possible. OpenBCI and InEar are not comparable — different hardware, different constraints.

5 Bandwidth Math

5.1 Serial 8-N-1 Basics

All three use serial (UART) 8-N-1. Each byte = 10 bits on wire:

$$\underbrace{1}_{\text{start}} + \underbrace{8}_{\text{data}} + \underbrace{0}_{\text{parity}} + \underbrace{1}_{\text{stop}} = 10 \text{ bits per byte}$$

Therefore:

$$\text{Max bytes/sec} = \frac{\text{Baud rate}}{10} \quad (\text{NOT } \div 8!)$$

5.2 Bandwidth Table

	OpenBCI Cyton	**InEar Original**	**InEar Optimized**
Baud rate	115,200	2,000,000	2,000,000
Max B/s (baud ÷ 10)	11,520	200,000	200,000
Packet size	33 B	56 B	26–54 B
Sample rate	250 Hz	1,000 Hz	1,000 Hz
Actual B/s	8,250	56,000	∼28,500
Wire bit rate	82,500 bps	560,000 bps	∼285,000 bps
Utilization	**71.6%**	**28.0%**	**14.3%**
Headroom	**28.4%** △	**72.0%** ✓	**85.7%** ✓

Table 7: Bandwidth utilization comparison

5.3 Headroom

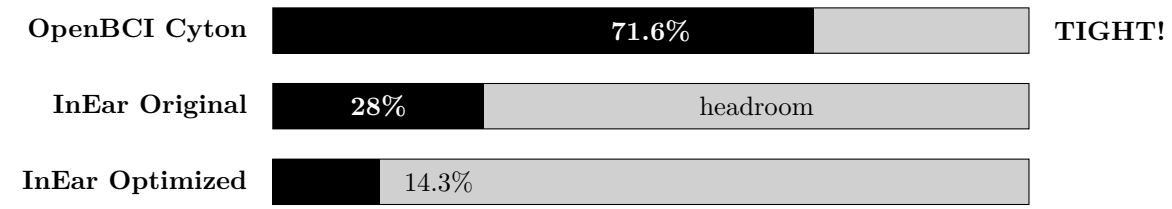

Figure 6: Bandwidth utilization comparison (bar chart). Each bar represents the percentage of the serial link capacity used.

5.4 Where The Bytes Go (Original)

Component	B/s	% of total
EEG (brain signals)	15,000	26.8%
PPG (heart pulse)	**18,000**	**32.1%**
Accelerometer	6,000	10.7%
Temp + Battery	4,000	7.1%
Sync	2,000	3.6%
Overhead (hdr+ts+mkr+ctr+chk+ftr)	11,000	19.6%
Total	**56,000**	

Table 8: Byte budget breakdown — InEar Original. PPG alone = 18,000 B/s = 32% of all traffic!

5.5 Where The Bytes Go (Optimized)

Component	B/s	% of total
EEG (every packet)	15,000	52.6%
Accel (every 4th)	1,500	5.3%
PPG (every 10th)	1,800	6.3%
Health (every 100th)	40	0.1%
Headers/footers	∼10,160	35.6%
Total	**∼28,500**	

Table 9: Byte budget breakdown — InEar Optimized. **Savings vs. original: ∼49%**

6 How The Stream Looks Over Time

6.1 How Packets Flow

Listing 1: Packet timing comparison

```
OpenBCI Cyton (250 Hz, uniform):
 --[33B]---[33B]---[33B]---[33B]---[33B]---  (4 ms apart)

InEar Original (1000 Hz, uniform):
 -[56]-[56]-[56]-[56]-[56]-[56]-[56]-[56]-  (1 ms apart, same every time)

InEar Optimized (1000 Hz, variable):
 -[26]-[26]-[26]-[32]-[26]-[26]-[26]-[32]-[26]-[50]-[26]-[26]-
   ^          ^              ^           ^
   |          |              |           +- EEG + accel + PPG (every 20th)
   |          |              +- EEG + accel (every 4th)
   |          +- EEG + accel (every 4th)
   +- EEG only (most common - small and fast!)
```

6.2 What 1 Second of Data Looks Like

In one second (1000 packets), the optimized protocol sends:

Packet Type	Size	Count/sec	% of packets
EEG-only	26 B	749	74.9%
EEG + Accel	32 B	200	20.0%
EEG + PPG	44 B	40	4.0%
EEG + Accel + PPG	50 B	9	0.9%
EEG + Health	30 B	∼1	0.1%
EEG + Full Sync	54 B	1	0.1%
Total		**1000**	

Table 10: Packet type distribution. 75% of packets are the small 26-byte EEG-only type.

6.3 What Happens When Data Is Missing?

When the optimized protocol sends an EEG-only packet (no accel), the receiver doesn't show a gap — it just **holds the last value**:

Listing 2: Sample-and-hold behavior for multi-rate sensors

```
Time:   1ms   2ms   3ms   4ms   5ms   6ms   7ms   8ms
EEG:    new   new   new   new   new   new   new   new   (every packet)
Accel:  new   held  held  NEW   held  held  held  NEW   (every 4th)
PPG:    new   held  held  held  held  held  held  held  (every 10th)
```

Accel hasn't changed in 4 ms. PPG hasn't changed in 10 ms. No information lost.

7 Original vs. Optimized — Tradeoffs

Dimension	InEar Original	InEar Optimized	Winner
Bandwidth	56 KB/s	∼28.5 KB/s	✓ Optimized (49% less)
Latency	280 μs wire	130–270 μs wire	✓ Optimized
Integrity	XOR checksum	XOR checksum	= Tie
Parser Complexity	Simple: fixed offsets	Complex: 14 types	✓ Original
Firmware Complexity	Simple: fill 56 B	Moderate: variable	✓ Original
Jitter Robustness	Excellent: uniform	Good: variable sizes	✓ Original
Packet Loss Robustness	Good: self-contained	Moderate: stale data	✓ Original
Headroom	72% free	86% free	✓ Optimized
Memory Efficiency	All bytes used	Zero overhead	= Tie
Extensibility	Fixed layout	New packet type needed	✓ Original

Table 11: Head-to-head trade-off comparison

8 All The Optimization Ideas (28 Tactics)

28 optimization ideas in 7 categories, starting from the 56-byte fixed protocol.

8.1 Category 1: Bandwidth Optimization

Fewer bytes on the wire

ID	Tactic	Description
1A ✓ Implemented	Multi-rate aux sampling	Send aux sensors only at their Nyquist rate
1B	Delta encoding for EEG	Send differences instead of absolute values
1C	Variable-width integers	Small values use fewer bytes
1E	PPG to 24-bit	PPG ADCs are 18–20 bit; 24-bit is plenty
1F	Smaller header	Reduce or compress header fields
1G	Multi-sample batching	Pack N samples per packet, amortize overhead

Table 12: Category 1: Bandwidth optimization tactics

8.2 Category 2: Latency Optimization

Faster ADC-to-screen time

ID	Tactic	Description
2A	Reduce packet size	Smaller packet = faster on wire (covered by Cat. 1)
2B	Increase baud rate	Teensy 4.1 supports up to 6 Mbaud
2C	USB flush optimization	Use `Serial.send_now()` for immediate transfer
2D	DMA SPI→UART pipeline	Hardware transfers without CPU involvement

Table 13: Category 2: Latency optimization tactics

8.3 Category 3: Integrity Optimization

Catching errors before they corrupt data

ID	Tactic	Description
3A	CRC-8 or CRC-16	Better error detection than XOR checksum
3B	Forward Error Correction	Correct errors without retransmission (Hamming/RS)
3C	Retransmission (ARQ)	Receiver requests re-send of bad packets
3D	Longer sync pattern	3–4 byte sync for fewer false positives
3E	Explicit length field	Self-describing packets, forward-compatible

Table 14: Category 3: Integrity optimization tactics

8.4 Category 4: Parser/CPU Optimization

Faster host-side parsing

ID	Tactic	Description
4A	Ring buffer	Replace `std::deque` with fixed circular buffer
4B	Zero-copy parsing	Parse directly from buffer, skip memcpy
4C	SIMD byte conversion	Parallel 24-bit → 32-bit conversion
4D	Remove `std::chrono`	Avoid syscalls during packet parsing
4E	Batch `addToBuffer()`	Fewer lock acquisitions in Open Ephys

Table 15: Category 4: Parser/CPU optimization tactics

8.5 Category 5: Firmware CPU Optimization

Less Teensy CPU usage

ID	Tactic	Description
5A	Precomputed templates	Pre-fill headers, only overwrite data fields
5B	Packet type lookup table	Replace modulo operations with array lookup
5C	ISR-based assembly	Build packets in interrupt handler

Table 16: Category 5: Firmware CPU optimization tactics

8.6 Category 6: Robustness Optimization

Reliable connections, no data loss

ID	Tactic	Description
6A	Heartbeat/watchdog	Detect disconnections immediately
6B	Connection handshake	Verify firmware version before streaming
6C	Configurable sample rate	Host can change rate (250/500/1000/2000 Hz)
6D	Per-channel gain control	Host can set ADS1299 gain per channel

Table 17: Category 6: Robustness optimization tactics

8.7 Category 7: Data Quality Optimization

Better data for research

ID	Tactic	Description
7A	Hardware clock sync	Sub-microsecond timing via host↔device sync
7B	Impedance measurement	Report electrode impedance in packets
7C	Higher sample rate	ADS1299 supports up to 16 kHz

Table 18: Category 7: Data quality optimization tactics

9 Detailed Notes On Each Tactic

9.1 Category 1 — Bandwidth

9.1.1 1A — Multi-Rate Auxiliary Sampling ✓ Implemented

Send each sensor at a rate matching how fast it actually changes (Nyquist rate), not all at 1000 Hz.

Sensor	BW of Interest	Nyquist Min	Send Rate	Oversampling
EEG	0.1–300 Hz	600 Hz	1000 Hz	1.67×
Accelerometer	0–100 Hz	200 Hz	250 Hz	1.25×
PPG	0.5–25 Hz	50 Hz	100 Hz	2×
Temperature	∼0 Hz (DC)	∼1 Hz	10 Hz	10×

Table 19: Nyquist justification for multi-rate scheduling

Why: Temperature changes over seconds. Sending it 1000×/sec = same reading every time.

Trade-off: Parser goes from 1 packet type to 14 (7 base × 2 with marker). Receiver needs sample-and-hold. Lost PPG packet = stale for 10 ms not 1 ms.

Impact: Saves ∼49% bandwidth (56,000 → ∼28,500 B/s).

9.1.2 1B — Delta Encoding for EEG

Send differences between consecutive samples instead of absolute values. At 1 kHz, EEG deltas fit in 8–12 bits vs. 24.

Listing 3: Delta encoding example

```
Absolute:  sample1 = 123456    -> 3 bytes (24-bit)
           sample2 = 123489    -> 3 bytes (24-bit)
           sample3 = 123501    -> 3 bytes (24-bit)

Delta:     sample1 = 123456    -> 3 bytes (24-bit, keyframe)
           d2 = +33            -> 1 byte  (8-bit delta)
           d3 = +12            -> 1 byte  (8-bit delta)

Savings: 9 bytes -> 5 bytes for 3 samples (44%)
```

Note to self: Losing one packet corrupts everything until the next keyframe. Need periodic absolute keyframes (e.g., every 100 samples) and gap detection via sequence numbers. Potential savings: 40–60% for EEG.

9.1.3 1C — Variable-Width Integers (VarInt)

Small values use fewer bytes (like Protocol Buffers).

Trade-off:

✗ Packets have **unpredictable size** — parser can't jump to fixed offsets
✗ Bit-level parsing is expensive
✗ Cannot pre-validate packet size
✓ Average savings of ∼30% combined with delta encoding

Verdict: High complexity, moderate savings. Better for file formats than real-time streams.

9.1.4 1E — PPG to 24-bit (from 48-bit) ✓ Free Win

PPG is 48-bit (6 bytes/channel) but the MAX30102 ADC is 18-bit. 24-bit (3 bytes) is enough.

$$\begin{aligned} \text{Current:} &\quad 3 \text{ channels} \times 6\,\text{B} = 18\,\text{B/sample} \\ \text{Proposed:} &\quad 3 \text{ channels} \times 3\,\text{B} = 9\,\text{B/sample} \\ \text{Savings:} &\quad 9\,\text{B per PPG packet} \end{aligned}$$

Firmware uses `packInt48BE()`, top 3 bytes always zero. 24-bit = 16.7M levels for ∼18 bits of data. Sufficient.

Trade-off: None. Free win.

9.1.5 1F — Smaller Header/Footer

Current: 2-byte header (`0xA5 0x5A`) + 2-byte footer (`0xC0 0xC0`) = 4 bytes/packet.

False sync math: With a 2-byte header, false sync probability is 1/65,536. With 1-byte: 1/256 — 256× more likely.

Verdict: Saves 1–3 B/pkt, makes resyncing harder. Not worth it.

9.1.6 1G — Multi-Sample Batching

Pack N samples per packet, share header/footer/checksum cost.

$$\text{Current (1/pkt):} \quad 3 \times 26 = 78\,\text{B for 3 samples, 33 B overhead (42\%)}$$
$$\text{Batched (3/pkt):} \quad 8 + 3 \times 15 + 3 = 56\,\text{B for 3 samples, 11 B overhead (20\%)}$$

Trade-off:
✗ Latency increases by $N\times$ (at 1 kHz with $N = 4$: 4 ms instead of 1 ms)
✗ Losing one packet loses N samples
✓ At $N = 4$, overhead drops from 42% to 17%
Verdict: Bad for real-time neurofeedback. OK for recording.

9.2 Category 2 — Latency

9.2.1 2A — Reduce Packet Size

Side effect of Category 1. Smaller packet = less wire time.

$$\text{Original (56 B):} \quad 56 \times 10/2{,}000{,}000 = 280\,\mu\text{s}$$
$$\text{Optimized (26 B):} \quad 26 \times 10/2{,}000{,}000 = 130\,\mu\text{s} \;\; (54\%\text{ faster})$$

9.2.2 2B — Increase Baud Rate

Teensy 4.1 supports up to 6 Mbaud. Currently at 2 Mbaud.

Baud Rate	Max B/s	Wire time (56 B)	Wire time (26 B)
2,000,000	200,000	280 μs	130 μs
3,000,000	300,000	187 μs	87 μs
4,000,000	400,000	140 μs	65 μs
6,000,000	600,000	93 μs	43 μs

Table 20: Wire time at various baud rates

Trade-off: Higher bit error rate, USB-serial IC may not support it, cable quality matters above 3 Mbaud. Low priority — 130 μs is small vs. 1 ms sample period.

9.2.3 2C — USB Flush Optimization

Call `Serial.send_now()` to flush USB immediately instead of waiting for the ∼1 ms default buffer interval.

Listing 4: USB flush optimization

```
Serial.write(packet, len);
Serial.send_now();  // Force immediate USB transfer
// USB packet sent on next USB frame (~125 us for USB 2.0 HS)
```

Verdict: One line of code. Removes up to 1 ms latency. Not implemented.

9.2.4 2D — DMA SPI→UART Pipeline

DMA hardware transfers data from ADS1299 SPI to UART without CPU.

Listing 5: CPU-mediated vs. DMA data flow

```
Current:  ADS1299 --SPI--> [CPU reads] --> [CPU builds] --> [CPU writes UART]
DMA:      ADS1299 --SPI--> [DMA->RAM]  --> [CPU builds] --> [DMA->UART]
                           ^ HW only ^                      ^ HW only ^
```

Verdict: High effort, high reward. Only useful if Teensy needs CPU for other tasks.

9.3 Category 3 — Integrity

9.3.1 3A — CRC-8 Instead of XOR ✓ Free Win

Same 1-byte cost as XOR, much better error detection:

Error Type	XOR	CRC-8
Single-bit errors	100%	100%
2-bit errors	100%	100%
Odd # of bit errors	100%	100%
Even # of bit errors	**0%!!**	99.6%
Burst errors ($\leq$8 bit)	$\sim$50%	100%
Random errors	$\sim$50%	99.6%

Table 21: Error detection: XOR vs. CRC-8

Needs a 256-byte lookup table and one lookup per byte. 56,000 lookups/sec on a 600 MHz Teensy = negligible.

9.3.2 3B — Forward Error Correction (FEC)

Add redundancy bytes to correct errors without retransmission (Hamming, Reed-Solomon).

Verdict: Overkill for wired USB. USB 2.0 has hardware CRC16. Only relevant for wireless.

9.3.3 3C — Retransmission (ARQ)

Receiver sends NACK on bad packets; firmware re-sends.

Trade-off: 2–3 ms latency per retry, needs bidirectional protocol and TX buffer on Teensy. Near-zero USB loss rate = not needed.

9.3.4 3D — Longer Sync Pattern

Use 3–4 byte sync instead of current 2-byte `0xA5 0x5A`.

$$\begin{aligned} \text{2-byte:} &\quad P(\text{false sync}) = 1/2^{16} = 1/65{,}536 \\ \text{3-byte:} &\quad P(\text{false sync}) = 1/2^{24} = 1/16{,}777{,}216 \\ \text{4-byte:} &\quad P(\text{false sync}) = 1/2^{32} \approx 1/4.3 \times 10^{9} \end{aligned}$$

Verdict: Current 2-byte + checksum is sufficient.

9.3.5 3E — Explicit Length Field

Add a 1-byte length field so the parser knows the exact packet size before reading.

Trade-off: +1 byte/packet (1,000 B/s). Host can skip unknown packet types without a lookup table.

Verdict: Good trade-off. Worth doing in any evolving protocol.

9.4 Category 4 — Parser/CPU (Host Side)

9.4.1 4A — Ring Buffer Instead of `std::deque` ✓ Free Win

Current parser uses `std::deque<uint8_t>`. Each `push_back()`/`pop_front()` may allocate heap. At 28,500 B/s = up to 28,500 allocs/sec, bad cache locality.

Fixed 16 KB circular buffer removes all allocations:

Listing 6: Ring buffer implementation sketch

```
class RingBuffer {
    uint8_t buffer[16384];  // 16 KB, power of 2
    size_t head = 0, tail = 0;
public:
    void push(const uint8_t* data, size_t len) {
        for (size_t i = 0; i < len; i++)
            buffer[(tail++) & 0x3FFF] = data[i];
    }
    uint8_t operator[](size_t i) const {
        return buffer[(head + i) & 0x3FFF];
    }
    void consume(size_t n) { head += n; }
    size_t available() const { return tail - head; }
};
```

No allocations in hot path. Better cache. Same behavior.

9.4.2 4B — Zero-Copy Parsing

Parse directly from ring buffer, skip the copy to a temp struct.

Trade-off: Must handle wrap-around mid-packet. Saves one memcpy/packet (56 B × 1000/sec = 56 KB/s eliminated). Best combined with 4A.

9.4.3 4C — SIMD Byte Conversion

SSE/AVX (x86) or NEON (ARM) for parallel 24-bit BE → 32-bit signed conversion.

Verdict: Only 5 channels. Sequential code takes nanoseconds. Not worth platform-specific code.

9.4.4 4D — Remove `std::chrono` from Hot Path ✓ Free Win

Parser calls `std::chrono::steady_clock::now()` every `updateBuffer()` for bandwidth monitoring. On Windows = kernel syscall.

Fix: Use packet counter (`if (count % 5000 == 0)`).

Effort: ∼10 min, 3–4 lines.

9.4.5 4E — Batch `addToBuffer()` Calls

1 call/packet = 1000 lock cycles/sec. Batch N samples = $1000/N$ locks.

Trade-off: Up to N ms extra latency. Good for throughput, bad for real-time.

9.5 Category 5 — Firmware CPU (Teensy Side)

9.5.1 5A — Precomputed Packet Templates

Pre-fill static fields (header, footer, type) at startup into 7 templates. In ISR: `memcpy` template, overwrite dynamic fields.

Trade-off: $7 \times 55 = 385$ bytes RAM. Teensy has 1 MB. Negligible.

9.5.2 5B — Packet Type Lookup Table ✓ Free Win

Replace cascading modulo with a precomputed 1000-entry array.

Listing 7: Packet type lookup table

```
// Current: 5 modulo operations + 5 branches
uint8_t getPacketType(uint32_t n) {
    if (n % 1000 == 0) return 0x06;
    if (n % 100 == 0)  return 0x04;
    if (n % 20 == 0)   return 0x03;
    if (n % 10 == 0)   return 0x02;
    if (n % 4 == 0)    return 0x01;
    return 0x00;
}

// Proposed: 1 modulo + 1 array read
static uint8_t typeTable[1000];  // precomputed at startup
uint8_t type = typeTable[sampleNum % 1000];
```

Uses 1000 bytes of RAM. Teensy has 1 MB. **Free win**.

9.5.3 5C — ISR-Based Packet Assembly

Build packets in IntervalTimer ISR for deterministic timing.

Status: Already implemented. Firmware builds and sends in ISR callback.

9.6 Category 6 — Robustness

9.6.1 6A — Heartbeat / Watchdog

Firmware sends "I'm alive" packet every ∼1 sec. No heartbeat for N seconds = connection dead.

Listing 8: Heartbeat disconnect detection

```
Normal:   Host <--- [Data] [Data] [HB] [Data] --- Teensy

Failure:  Host <--- [Data] [Data]             --- (cable yanked!)
          ...3 seconds, no heartbeat...
          HOST: CONNECTION LOST -> show error, clean up
```

Verdict: ∼10 B/s overhead. Easy to add.

9.6.2 6B — Connection Handshake

Startup: `HELLO` → Device Info → `START` → streaming → `STOP` → `OK`.

Auto-detects firmware version, channel count, sample rate. Prevents mismatch.

Trade-off: +∼100 ms startup. Negligible.

9.6.3 6C — Configurable Sample Rate

Host sets Teensy sample rate at runtime.

Rate	Use Case
250 Hz	Low power, clinical EEG
500 Hz	Balanced research EEG
1000 Hz	High-frequency EEG (current default)
2000 Hz	Fast neural events
4000 Hz	EMG, specialized

Table 22: ADS1299 configurable sample rates

Verdict: Useful. Pairs with handshake (6B). Moderate effort.

9.6.4 6D — Per-Channel Gain Control

Set ADS1299 PGA gain per channel at runtime. Gains: 1×, 2×, 4×, 6×, 8×, 12×, 24×.

Use case: Channels 1–4 at 24× (EEG) and channel 5 at 1× (EMG for jaw clench detection).

9.7 Category 7 — Data Quality

9.7.1 7A — Hardware Clock Synchronization

Sync Teensy crystal with host clock. Without it: ∼50 ppm drift → 180 ms off after 1 hour.

Method: Periodic NTP-style time exchange (T_1, T_2, T_3, T_4):

$$\text{Offset} = \frac{(T_2 - T_1) + (T_3 - T_4)}{2}, \qquad \text{RTT} = (T_4 - T_1) - (T_3 - T_2)$$

After correction: ∼100 μs accuracy. Required for multi-device setups.

9.7.2 7B — Impedance Measurement

Use ADS1299's built-in impedance mode to check electrode contact quality.

Impedance	Quality	Interpretation
$< 10\,\text{k}\Omega$	✓ Good	Gel electrodes, well-placed
10–50 kΩ	△ Acceptable	Dry electrodes, may work
$> 50\,\text{k}\Omega$	✗ Poor	Needs adjustment
$> 1\,\text{M}\Omega$	✗ None	No electrode detected

Table 23: Electrode impedance quality thresholds

Verdict: Useful for research. Setup-time only, not during streaming.

9.7.3 7C — Higher Sample Rate

ADS1299 supports up to 16 kHz. Bandwidth implications:

Rate	B/s (opt)	Utilization	Status
250 Hz	∼7,125	3.6%	OK
500 Hz	∼14,250	7.1%	OK
1,000 Hz	∼28,500	14.3%	Current
2,000 Hz	∼57,000	28.5%	OK
4,000 Hz	∼114,000	57.0%	Tight
8,000 Hz	∼228,000	114%	**Exceeds 2 Mbaud!**
16,000 Hz	∼456,000	228%	**Need 6 Mbaud+**

Table 24: Sample rate vs. bandwidth utilization

Current protocol supports up to ∼4000 Hz within the 2 Mbaud link budget.

10 The Big Trade-Off Table

▼ = good (reduces/improves), ▲ = bad (increases/worsens), = = doesn't matter.

Table 25: Full 28-tactic trade-off matrix

Tactic	BW	Latency	Integrity	Parser	FW CPU	Complex.	Robust.
1A Multi-rate	▼▼ 49%	▼ smaller	=	▲▲ state mach.	▲ modulo	▲▲ 14 types	▼ stale risk
1B Delta enc.	▼▼▼ 60%	▼ smaller	▼▼▼ propagates	▲▲ accum.	▲ delta math	▲▲▲ keyframes	▼▼▼ corrupts
1C VarInt	▼▼ 30%	▼ smaller	▼ no pre-val	▲▲▲ bit parse	▲ encoding	▲▲▲ no fixed	▼▼ unpredict.
1E PPG 24-bit	▼ 9 B/PPG	▼ 9 B less	=	▼ reuse parser	▼ simpler	▼ fewer widths	=
1F Sm. header	▼ 2–4 B	▼ slight	▼ less resync	=	▼ fewer B	▲ tighter	▼ less recov.
1G Batch N	▼▼ amortize	▲▲ $N\times1$ ms	▲ lose N	▲ unpack	▲ buffer	▲▲ edge cases	▼▼ more loss
2A Reduce size	▼ (via Cat 1)	▼ fewer bits	=	=	=	=	=
2B Higher baud	=	▼ faster wire	▼ higher BER	=	= config	= config	▼ bit errors
2C send_now()	=	▼ 0.5 ms	=	=	▲ USB IRQs	▲ ISR timing	=
2D DMA	=	▼▼ no CPU	=	=	▼▼ freed	▲▲ setup	▲ HW xfer
3A CRC-8	= same	=	▼▼ 99.6%	▲ table lkup	▲ table lkup	▲ 256 B table	▲ fewer undet.
3B FEC	▲▲ 20–50%	▲ enc/dec	▼▼▼ corrects	▲▲▲ decode	▲▲▲ encode	▲▲▲ overkill	▲▲ corrected
3C Retransmit	▲ NACK	▲▲ round-trip	▼▼ guaranteed	▲▲ reorder	▲▲ TX buf	▲▲▲ bidir.	▲▲▲ no loss
3D Longer sync	▲ +1–2 B	=	▲ lower false	=	=	=	▲ reliable
3E Length field	▲ +1 B	=	▲ validate	▼ no LUT	▲ +1 byte	▼ fwd-compat	▲ new types
4A Ring buffer	=	▼ alloc jitter	=	▼▼ no heap	=	▲ wraparound	▲ determin.
4B Zero-copy	=	▼ less memcpy	=	▼ no copy	=	▲ wrap parse	=
4C SIMD	=	▼ faster cvt	=	▼ parallel	=	▲▲ platform	=
4D Rm. chrono	=	▼ fewer sys	=	▼ no syscall	=	▼ simpler	▲ determin.
4E Batch addTo	=	▲ batch delay	=	▼ fewer locks	=	▲ batch code	=
5A Templates	=	=	=	=	▼ fewer wr.	▼ cleaner	=
5B Type LUT	=	=	=	=	▼ 1 vs 5 mod	▼ simpler	=
5C ISR assembly	=	▼ no loop	=	=	▲ longer ISR	▲ overrun	▲ determin.
6A Heartbeat	▲ ∼1 pkt/s	=	=	▲ timeout	▲ HB gen	▲ state mach.	▲▲ disconnect
6B Handshake	▲ HS bytes	▲ 100 ms	▲ ver. verify	▲ HS FSM	▲ cmd parser	▲▲ bidir.	▲▲ ver. match
6C Config rate	= steady	=	=	▲ handle chg	▲ timer cfg	▲▲ cmd proto	▲ adaptable
6D Gain ctrl	= steady	=	=	▲ track gain	▲ ADS writes	▲▲ cmd proto	▲ adapt sig.
7A Clock sync	▲ 2–4 B	=	=	▲ drift math	▲ sync resp.	▲▲ sync proto	▲ drift det.
7B Impedance	▲ sync bytes	=	=	▲ display	▲▲ ADS mode	▲▲ mode sw.	▲▲ bad elec.
7C Higher rate	▲▲ linear	▼ temporal	=	▲ more samp.	▲ tighter	▲ headroom	=

11 Easy Wins — Do These First

Improvements with no downside:

Priority	ID	Tactic	Effort	Benefit
1st	4D	Remove `std::chrono`	10 min	Eliminates 1–3 syscalls per packet from parsing loop
1st	4A	Ring buffer	1 hr	Eliminates ∼28,500 heap allocs/sec, better cache
1st	1E	PPG to 24-bit	30 min	Saves 9 B per PPG packet, no resolution loss
2nd	3A	CRC-8	30 min	Same 1-byte cost, 99.6% burst error detection (vs. 50%)
2nd	5B	Packet type LUT	15 min	Replace 5 modulo + branch with 1 array lookup
2nd	5A	Precomputed templates	20 min	Pre-fill headers at startup, saves writes/sample
3rd	4E	Batch addToBuffer	30 min	Reduce lock contention in Open Ephys pipeline
3rd	3E	Explicit length field	30 min	+1 B cost, protocol becomes self-describing

Table 26: Recommended free-win optimizations, prioritized by effort-to-benefit ratio

12 Comparing With OpenBCI Isn't Really Fair

Different problems, different hardware:

	OpenBCI Cyton	InEar Teensy
Link	Wireless (2.4 GHz radio)	Wired (USB)
Bottleneck	Radio BW (31 B/pkt max)	Effectively unlimited
EEG channels	8	5
Sensor suite	EEG + accel only	EEG + accel + PPG + temp + batt + sync
Primary use	General research EEG	Specialized in-ear wearable
Baud rate	115,200 (radio-limited)	2,000,000 (USB-capable)
Sample rate	250 Hz (radio-limited)	1,000 Hz

Table 27: Fundamental hardware differences between OpenBCI and InEar

Note to self: OpenBCI 71.6% utilization = at the physical limit of its radio. InEar has headroom because USB is much faster than 2.4 GHz radio.

If OpenBCI had InEar's sensors, it would need $> 4\times$ its bandwidth. Not possible without hardware change.

13 Reminder: Serial 8-N-1 Bandwidth Math

Common mistake: `baud_rate / 8` instead of `baud_rate / 10`.

13.1 What "8-N-1" Actually Means

$$
\begin{aligned}
8 &= 8 \text{ data bits per character} \\
\text{N} &= \text{No parity bit} \\
1 &= 1 \text{ stop bit}
\end{aligned}
$$

$$
\text{Total:} \quad \underbrace{1}_{\text{start}} + \underbrace{8}_{\text{data}} + \underbrace{0}_{\text{parity}} + \underbrace{1}_{\text{stop}} = 10 \text{ bits per byte on the wire}
$$

13.2 The Formula

$$
\text{Bytes/sec} = \frac{\text{Baud rate}}{10}
$$

Baud Rate	**Correct (÷10)**	**Wrong (÷8)**
115,200	11,520 B/s	14,400 B/s
2,000,000	200,000 B/s	250,000 B/s

Table 28: Correct vs. incorrect serial throughput calculations

13.3 Utilization

$$
\textbf{OpenBCI Cyton:} \quad \frac{33 \times 250}{115{,}200/10} = \frac{8{,}250}{11{,}520} = \mathbf{71.6}\% \tag{1}
$$

$$
\textbf{InEar Original:} \quad \frac{56 \times 1000}{2{,}000{,}000/10} = \frac{56{,}000}{200{,}000} = \mathbf{28.0}\% \tag{2}
$$

$$
\textbf{InEar Optimized:} \quad \frac{\sim 28{,}500}{2{,}000{,}000/10} = \frac{28{,}500}{200{,}000} = \mathbf{14.3}\% \tag{3}
$$

14 Reference: Exact Byte Offsets For Every Packet

Exact byte offset, size, and format of every field in every packet. Use this when writing parsers — each row tells you exactly where in the raw bytes to find each value.

14.1 OpenBCI Cyton — 33 Bytes (Fixed)

Table 29: OpenBCI Cyton byte-level packet layout

Offset	Size	Field	Format	Description
0	1 B	Header	`0xA0`	Sync byte (constant)
1	1 B	Sample #	uint8	Counter 0–255 (wraps)
2–4	3 B	EEG Ch 1	int24 BE	Channel 1, 24-bit signed
5–7	3 B	EEG Ch 2	int24 BE	Channel 2, 24-bit signed
8–10	3 B	EEG Ch 3	int24 BE	Channel 3, 24-bit signed
11–13	3 B	EEG Ch 4	int24 BE	Channel 4, 24-bit signed
14–16	3 B	EEG Ch 5	int24 BE	Channel 5, 24-bit signed
17–19	3 B	EEG Ch 6	int24 BE	Channel 6, 24-bit signed
20–22	3 B	EEG Ch 7	int24 BE	Channel 7, 24-bit signed
23–25	3 B	EEG Ch 8	int24 BE	Channel 8, 24-bit signed
26–31	6 B	Aux Data	varies	Format depends on footer byte
32	1 B	Footer	`0xC0`–`0xC6`	Packet type indicator
	Total:	**33 bytes**		

Table 30: OpenBCI Cyton auxiliary data interpretation (bytes 26–31)

Footer	Aux Layout (bytes 26–31)	Description
`0xC0`	AccX[2] | AccY[2] | AccZ[2]	Accelerometer, 3×16-bit signed BE
`0xC1`–`0xC4`	Timestamp & accel (firmware v2+)	
`0xC5`	RAW[6]	Raw user-defined aux bytes
`0xC6`	0x00 × 6	Unused (zero-filled)

14.2 InEar Teensy Original — 56 Bytes (Fixed)

Table 31: InEar Original byte-level packet layout

Offset	Size	Field	Format	Description
0	1 B	Sync 1	`0xA5`	Header byte 1 (constant)
1	1 B	Sync 2	`0x5A`	Header byte 2 (constant)
2–5	4 B	Timestamp	uint32 BE	Microsecond timestamp
6	1 B	Marker	uint8	Event marker byte
7–9	3 B	EEG Ch 1	int24 BE	Channel 1, 24-bit signed
10–12	3 B	EEG Ch 2	int24 BE	Channel 2, 24-bit signed
13–15	3 B	EEG Ch 3	int24 BE	Channel 3, 24-bit signed
16–18	3 B	EEG Ch 4	int24 BE	Channel 4, 24-bit signed
19–21	3 B	EEG Ch 5	int24 BE	Channel 5, 24-bit signed
22–23	2 B	Acc X	int16 BE	Accelerometer X
24–25	2 B	Acc Y	int16 BE	Accelerometer Y
26–27	2 B	Acc Z	int16 BE	Accelerometer Z
28–33	6 B	PPG Red	int48 BE	PPG Red LED (48-bit)
34–39	6 B	PPG IR	int48 BE	PPG IR LED (48-bit)
40–45	6 B	PPG Green	int48 BE	PPG Green LED (48-bit)
46–47	2 B	Temp	int16 BE	Temperature (0.01°C)
48–49	2 B	Battery	uint16 BE	Battery voltage (mV)
50–51	2 B	Sync	uint16 BE	External trigger
52	1 B	Pkt Counter	uint8	Packet counter (0–255)
53	1 B	Checksum	uint8	XOR of bytes 0–52
54	1 B	Footer 1	`0xC0`	End byte 1 (constant)
55	1 B	Footer 2	`0xC0`	End byte 2 (constant)
	Total:	**56 bytes**		

14.3 InEar Teensy Optimized — Variable Length

The optimized protocol has a **fixed 8-byte header** and **fixed 3-byte footer**, with variable data sections between them depending on the packet type.

Header (always present — 8 bytes)

Table 32: Optimized protocol header (all packet types)

Offset	Size	Field	Format	Description
0	1 B	Sync 1	`0xA5`	Header byte 1 (constant)
1	1 B	Sync 2	`0x5A`	Header byte 2 (constant)
2	1 B	Packet Type	uint8 enum	Content descriptor (see below)
3	1 B	Sequence	uint8	Counter 0–255 (wraps)
4–7	4 B	Timestamp	uint32 BE	Microseconds since boot
	Total:	**8 bytes**		

EEG Data (always present — 15 bytes)

Table 33: EEG data block (present in every packet)

Offset	Size	Field	Format	Description
8	3 B	EEG Ch 1	int24 BE	Channel 1, 24-bit signed
11	3 B	EEG Ch 2	int24 BE	Channel 2, 24-bit signed
14	3 B	EEG Ch 3	int24 BE	Channel 3, 24-bit signed
17	3 B	EEG Ch 4	int24 BE	Channel 4, 24-bit signed
20	3 B	EEG Ch 5	int24 BE	Channel 5, 24-bit signed
	Total:	**15 bytes** (offsets 8–22)		

Optional Data Blocks (variable offset)

These blocks come **after the EEG data** in this order. Their absolute offset depends on which blocks before them are present.

Table 34: Optional marker block (+1 byte). Present when type has `0x10` flag set.

Offset*	Size	Field	Format	Description
23*	1 B	Marker	uint8	Experiment event trigger

*Offset is 23 when marker is the first optional block

Table 35: Optional accelerometer block (+6 bytes). Types: `0x01`, `0x03`, `0x05`, `0x06`.

Relative	Size	Field	Format	Description
+0	2 B	Accel X	int16 BE	X-axis acceleration
+2	2 B	Accel Y	int16 BE	Y-axis acceleration
+4	2 B	Accel Z	int16 BE	Z-axis acceleration
	Total:	**6 bytes**		

Table 36: Optional PPG block (+18 bytes). Types: `0x02`, `0x03`, `0x06`.

Relative	Size	Field	Format	Description
+0	6 B	PPG Red	uint48 BE	Red LED, 48-bit unsigned
+6	6 B	PPG IR	uint48 BE	IR LED, 48-bit unsigned
+12	6 B	PPG Green	uint48 BE	Green LED, 48-bit unsigned
	Total:	**18 bytes**		

Table 37: Optional health block (+4 bytes). Types: `0x04`, `0x05`, `0x06`.

Relative	Size	Field	Format	Description
+0	2 B	Temperature	int16 BE	Body temp (0.01°C units)
+2	2 B	Battery	uint16 BE	Battery voltage (mV)
	Total:	**4 bytes**		

Footer (always present — 3 bytes)

Table 38: Optimized protocol footer (all packet types)

Offset	Size	Field	Format	Description
$n-3$	1 B	Checksum	uint8	XOR of bytes 0 to $n-4$
$n-2$	1 B	Footer 1	`0xC0`	End byte 1 (constant)
$n-1$	1 B	Footer 2	`0xC0`	End byte 2 (constant)
Total:	**3 bytes** (n = total packet length)			

Cheat Sheet: Offsets Per Packet Type

Here are the **absolute byte offsets** for every packet type — just look up the row and column:

Table 39: Absolute offset map for each optimized packet type (no marker)

Block	`0x00` EEG	`0x01` EEG+Acc	`0x02` EEG+PPG	`0x03` EEG+A+P	`0x04` EEG+Hlt	`0x05` EEG+A+H	`0x06` Full
Header	0–7	0–7	0–7	0–7	0–7	0–7	0–7
EEG	8–22	8–22	8–22	8–22	8–22	8–22	8–22
Accel	—	23–28	—	23–28	—	23–28	23–28
PPG	—	—	23–40	29–46	—	—	29–46
Health	—	—	—	—	23–26	29–32	47–50
Checksum	23	29	41	47	27	33	51
Footer	24–25	30–31	42–43	48–49	28–29	34–35	52–53
Total	**26 B**	**32 B**	**44 B**	**50 B**	**30 B**	**36 B**	**54 B**

Note to self: To read accelerometer Y from a Type `0x03` (EEG+Accel+PPG) packet: go to the Accel row → offsets 23–28 → Accel Y is at relative +2 within the block → **absolute offset 25–26**, 16-bit signed big-endian.

15 Reference: Tactic Quick-Reference Table

Same matrix as before, just in a compact format for quick lookups.

▼ = good (reduces/improves), ▲ = bad (increases/worsens), = = doesn't matter. More arrows = stronger effect.

Table 40: All 28 optimization tactics — trade-off matrix

Tactic	BW	Latency	Integrity	Parser	FW CPU	Complex.	Robust.
Category 1 — Bandwidth Optimization							
1A Multi-rate	▼▼ 49%	▼ smaller	=	▲▲ state mach.	▲ modulo	▲▲ 14 types	▼ stale risk
1B Delta enc.	▼▼▼ 60%	▼ smaller	▼▼▼ propagates	▲▲ accum.	▲ delta math	▲▲▲ keyframes	▼▼▼ corrupts
1C VarInt	▼▼ 30%	▼ smaller	▼ no pre-val	▲▲▲ bit parse	▲ encoding	▲▲▲ no fixed	▼▼ unpredict.
1E PPG 24-bit	▼ 9 B/PPG	▼ 9 B less	=	▼ reuse parser	▼ simpler	▼ fewer widths	=
1F Sm. header	▼ 2–4 B	▼ slight	▼ less resync	=	▼ fewer B	▲ tighter	▼ less recov.
1G Batch N	▼▼ amortize	▲▲ $N\times1$ ms	▲ lose N	▲ unpack	▲ buffer	▲▲ edge cases	▼▼ more loss
Category 2 — Latency Optimization							
2A Reduce size	▼ (via Cat 1)	▼ fewer bits	=	=	=	=	=
2B Higher baud	=	▼ faster wire	▼ higher BER	=	= config	= config	▼ bit errors
2C send_now()	=	▼ 0.5 ms	=	=	▲ USB IRQs	▲ ISR timing	=
2D DMA	=	▼▼ no CPU	=	=	▼▼ freed	▲▲ setup	▲ HW xfer
Category 3 — Integrity Optimization							
3A CRC-8	= same	=	▼▼ 99.6%	▲ table lkup	▲ table lkup	▲ 256 B table	▲ fewer undet.
3B FEC	▲▲ 20–50%	▲ enc/dec	▼▼▼ corrects	▲▲▲ decode	▲▲▲ encode	▲▲▲ overkill	▲▲ corrected
3C Retransmit	▲ NACK	▲▲ round-trip	▼▼ guaranteed	▲▲ reorder	▲▲ TX buf	▲▲▲ bidir.	▲▲▲ no loss
3D Longer sync	▲ +1–2 B	=	▲ lower false	=	=	=	▲ reliable
3E Length field	▲ +1 B	=	▲ validate	▼ no LUT	▲ +1 byte	▼ fwd-compat	▲ new types
Category 4 — Parser/CPU Optimization							
4A Ring buffer	=	▼ alloc jitter	=	▼▼ no heap	=	▲ wraparound	▲ determin.
4B Zero-copy	=	▼ less memcpy	=	▼ no copy	=	▲ wrap parse	=
4C SIMD	=	▼ faster cvt	=	▼ parallel	=	▲▲ platform	=
4D Rm. chrono	=	▼ fewer sys	=	▼ no syscall	=	▼ simpler	▲ determin.
4E Batch addTo	=	▲ batch delay	=	▼ fewer locks	=	▲ batch code	=
Category 5 — Firmware CPU Optimization							
5A Templates	=	=	=	=	▼ fewer wr.	▼ cleaner	=
5B Type LUT	=	=	=	=	▼ 1 vs 5 mod	▼ simpler	=
5C ISR assembly	=	▼ no loop	=	=	▲ longer ISR	▲ overrun	▲ determin.
Category 6 — Robustness Optimization							
6A Heartbeat	▲ ∼1 pkt/s	=	=	▲ timeout	▲ HB gen	▲ state mach.	▲▲ disconnect

continued on next page...

Tactic	BW	Latency	Integrity	Parser	FW CPU	Complex.	Robust.
6B Handshake	▲ HS bytes	▲ 100 ms	▲ ver. verify	▲ HS FSM	▲ cmd parser	▲▲ bidir.	▲▲ ver. match
6C Config rate	= steady	=	=	▲ handle chg	▲ timer cfg	▲▲ cmd proto	▲ adaptable
6D Gain ctrl	= steady	=	=	▲ track gain	▲ ADS writes	▲▲ cmd proto	▲ adapt sig.
Category 7 — Data Quality Optimization							
7A Clock sync	▲ 2–4 B	=	=	▲ drift math	▲ sync resp.	▲▲ sync proto	▲ drift det.
7B Impedance	▲ sync bytes	=	=	▲ display	▲▲ ADS mode	▲▲ mode sw.	▲▲ bad elec.
7C Higher rate	▲▲ linear	▼ temporal	=	▲ more samp.	▲ tighter	▲ headroom	=